# El contenedor `map`

## Un diccionario ordenado por clave

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Octubre 2026

# Objetivos de aprendizaje I

## Al finalizar esta sesión, el estudiante podrá

- Usar `map<K, V>` para insertar con `[]`, consultar con `count` y `find`, eliminar con `erase` y recorrer las claves de menor a mayor con un iterador (OA3).
- Seguir a mano el contenido de un `map` tras una secuencia de operaciones, incluida la inserción que hace `[]` cuando solo se quería consultar (OA2).
- Dar el costo de cada operación del `map` y compararlo con el de la tabla hash con encadenamiento (OA1).
- Escoger entre la tabla hash y el `map` según lo que pide un problema, y resolver con ellos problemas de juez (OA5).

# Contenido I


1 Repaso: la tabla hash con encadenamiento

2 Un problema donde importa el orden

3 El contenedor `map`

4 Lo que cuesta

5 Para el juez

6 Ejercicios

# Contenido I


1 Repaso: la tabla hash con encadenamiento

2 Un problema donde importa el orden

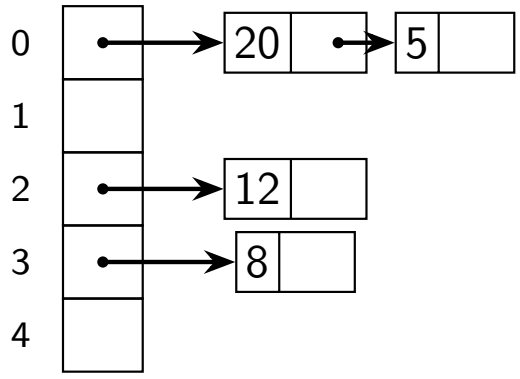
3 El contenedor `map`

4 Lo que cuesta

5 Para el juez

6 Ejercicios

# Lo que quedó de la tabla hash I




- $h(k) = k$ mód $m$ lleva cada clave a una de $m$ casillas; cada casilla es la cabeza de una lista enlazada.
- Insertar al frente cuesta $\Theta(1)$. Buscar y eliminar recorren una lista: $\Theta(1 + \alpha)$ en promedio, con $\alpha = n/m$, y $\Theta(n)$ en el peor caso.
- Espacio: $\Theta(m + n)$.

**Pregunta**

La tabla de la izquierda tiene $m = 5$ y guarda 5, 20, 12 y 8. ¿Cómo se imprimen esas claves de menor a mayor?

# Contenido I


1 Repaso: la tabla hash con encadenamiento

2 Un problema donde importa el orden

3 El contenedor `map`

4 Lo que cuesta

5 Para el juez

6 Ejercicios

# Un inventario I

**Problema**

Una tienda anota cada entrada de mercancía con el código del producto y la cantidad que llegó. Al cerrar el día imprime, por cada código, el total recibido, de menor a mayor código. Los códigos llegan hasta $10^9$ y en un día entran unos pocos miles de registros.

# ¿Entendimos el problema? I

## Entradas

$n$ y después $n$ registros, cada uno con un código y una cantidad.

| Código | Cantidad |
|---|---|
| 305 | 4 |
| 112 | 2 |
| 305 | 1 |
| 87 | 6 |
| 112 | 3 |

## Salidas

Una línea por código distinto con su total, en orden creciente de código.

| Código | Total |
|---|---|
| 87 | 6 |
| 112 | 5 |
| 305 | 5 |

# ¿Entendimos el problema? II

**Pregunta**

El 87 llegó de cuarto. ¿Por qué sale de primero? ¿Y por qué no sirve una tabla de direccionamiento directo?

# ¿Entendimos el problema? III

**Respuesta**

- La salida va por código, no por orden de llegada: 87 es el menor de los tres.
- Con códigos hasta $10^9$, la tabla directa pide $10^9$ casillas para unos pocos miles de códigos.
- Cada registro hace dos cosas: buscar el código y, si no estaba, agregarlo. Es el trabajo de la tabla hash.

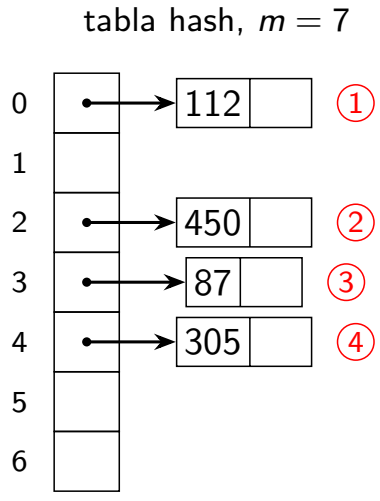
# Primera idea: la tabla hash I

Sumar las cantidades es buscar e insertar: $\Theta(1 + \alpha)$ en promedio por registro. Falta imprimir en orden.

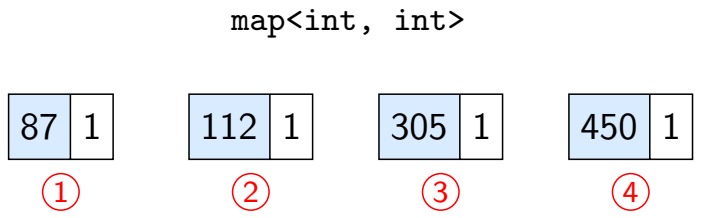
**Pregunta**

Con $m = 7$ se insertan 305, 112, 87 y 450, y después se recorre la tabla casilla por casilla con `imprimir`. ¿En qué orden salen?

# Primera idea: la tabla hash II




recorrido: 112 450 87 305




recorrido: 87 112 305 450

305 mód 7 = 4, 112 mód 7 = 0, 87 mód 7 = 3 y 450 mód 7 = 2. Los círculos dan el orden en que sale cada clave.

**La tabla hash ordena por casilla, no por clave**

El recorrido sigue el orden de $h(k)$. Que 112 quede en la casilla 0 y 87 en la 3 no dice nada sobre cuál clave es menor: la función hash reparte las claves y en el camino deshace su orden.

# Primera idea: la tabla hash III

```
int main() {
  int claves[4] = {305, 112, 87, 450};
  TablaHash t(7);
  map<int, int> m;
  int i = 0;
  while (i < 4) {
    t.insertar(claves[i]);
    m[claves[i]] = 1;
    i = i + 1;
  }
  printf("tabla hash, casilla por casilla:\n");
  t.imprimir();
  printf("map, con su iterador:");
  map<int, int>::iterator it = m.begin();
  while (it != m.end()) {
    printf(" %d", it->first);
    ++it;
  }
  printf("\n");
  return 0;
}
```

# Primera idea: la tabla hash IV

**Salida**

```
0: 112, 1:, 2: 450, 3: 87, 4: 305, 5:, 6:
map, con su iterador: 87 112 305 450
```

La última línea viene del contenedor de esta sesión, que recorre las mismas cuatro claves de menor a mayor.

**Arreglarlo al final**

Se copian las $n$ claves a un arreglo y se ordena: $\Theta(m+n)$ para recorrer y $\Theta(n \log n)$ para ordenar. Sirve si el orden se pide una vez. Si se pide tras cada registro, cada pedido vuelve a pagar el ordenamiento.

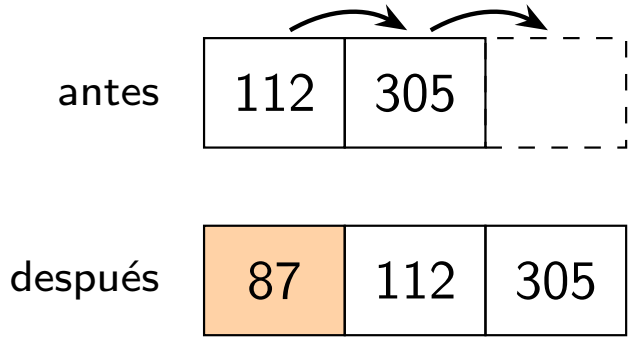
# Segunda idea: un vector ordenado I

Se guardan los códigos en un vector ordenado. Buscar es una búsqueda binaria, $\Theta(\log n)$, y recorrer en orden es leer el vector.

**Pregunta**

El vector tiene 112 305. ¿Qué pasa al llegar el 87?

# Segunda idea: un vector ordenado II




El 87 va adelante de todos: los dos códigos se corren una casilla.
Con $n$ códigos, insertar al frente corre los $n$: $\Theta(n)$.

| | Buscar | Insertar un código nuevo | Recorrer en orden |
|---|---|---|---|
| Tabla hash | $\Theta(1+\alpha)$ promedio | $\Theta(1+\alpha)$ promedio | $\Theta(m+n)+\Theta(n\log n)$ |
| Vector ordenado | $\Theta(\log n)$ | $\Theta(n)$ | $\Theta(n)$ |
| Lo que se quiere | $O(\log n)$ | $O(\log n)$ | $\Theta(n)$ |

# Contenido I


1 Repaso: la tabla hash con encadenamiento

2 Un problema donde importa el orden

3 El contenedor `map`

4 Lo que cuesta

5 Para el juez

6 Ejercicios

# Un diccionario ordenado I

## Definición

Un `map<K, V>` guarda parejas (*clave*, *valor*) con claves distintas, siempre en orden creciente de clave. Se busca por clave, como en la tabla hash, y se recorre de la menor clave a la mayor, como un vector ordenado.

## El inventario, tras los cinco registros

| 87 | 6 |
|---|---|

| 112 | 5 |
|---|---|

| 305 | 5 |
|---|---|

Las claves van en celeste y los valores en blanco. Hay tres parejas, una por código, y salen en el orden en que se imprimen.

# Un diccionario ordenado II

- `K` es el tipo de la clave y `V` el del valor: `map<int, int>` para código y total, `map<string, int>` para palabra y cuenta.
- La clave tiene que poderse comparar con `<`. El `map` no pide una función hash.
- La biblioteca lo guarda en un árbol balanceado, una estructura que el curso construye más adelante [1]. De ella salen los costos de esta sesión.

```
#include <map>
using std::map;

map<int, int> inv;          // vacio: size() vale 0
```

# El contrato I

| Operación | Qué hace |
|---|---|
| `m[k]` | el valor de `k`; si `k` no está, inserta $(k, 0)$ y devuelve ese 0 |
| `m[k] = v` | inserta $(k, v)$, o cambia el valor si `k` ya estaba |
| `m.count(k)` | 1 si `k` está, 0 si no; no inserta |
| `m.find(k)` | iterador a la pareja de `k`, o `m.end()` si no está |
| `m.erase(k)` | saca la pareja de `k`; devuelve cuántas sacó, 1 o 0 |
| `m.size()`, `m.empty()` | cuántas parejas hay; si no hay ninguna |
| `m.begin()`, `m.end()` | iterador a la menor clave; iterador después de la mayor |
| `it->first`, `it->second` | la clave y el valor de la pareja a la que apunta `it` |

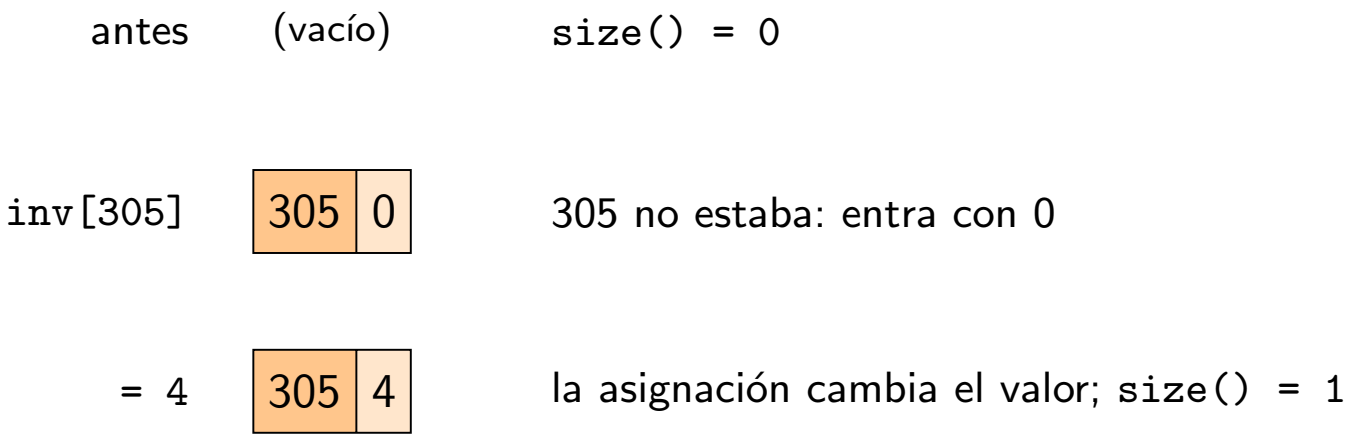
# El contrato II

**La clave no se cambia**

`it->second` se puede modificar; `it->first` no. Cambiar una clave en su sitio rompería el orden. Para cambiar una clave se saca la pareja y se inserta otra.

# Insertar con []: inv[305] = 4 I

```
map<int, int> inv;        // vacio: size() = 0
inv[305] = 4;
```

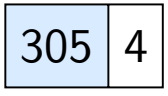



`inv[305] = 4` hace dos cosas: `inv[305]` busca 305 y, al no hallarlo, lo inserta con valor 0; después la asignación pone el 4.

# Insertar con []: `inv[112] = 2` I

```
inv[112] = 2;
```

antes



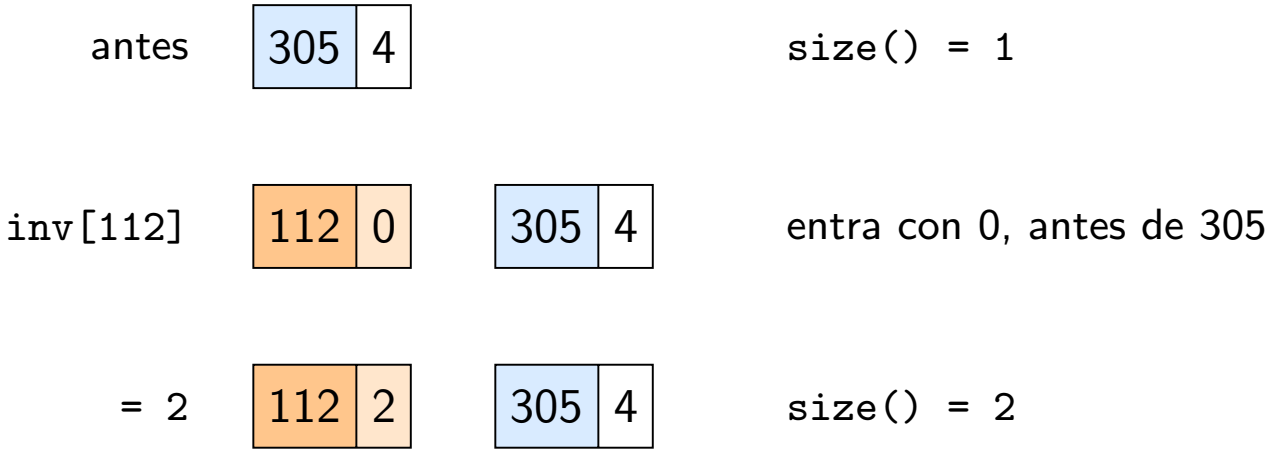

`size() = 1`

**Pregunta**

112 llega después de 305. ¿Dónde queda, antes o después?

# Insertar con []: `inv[112] = 2` II

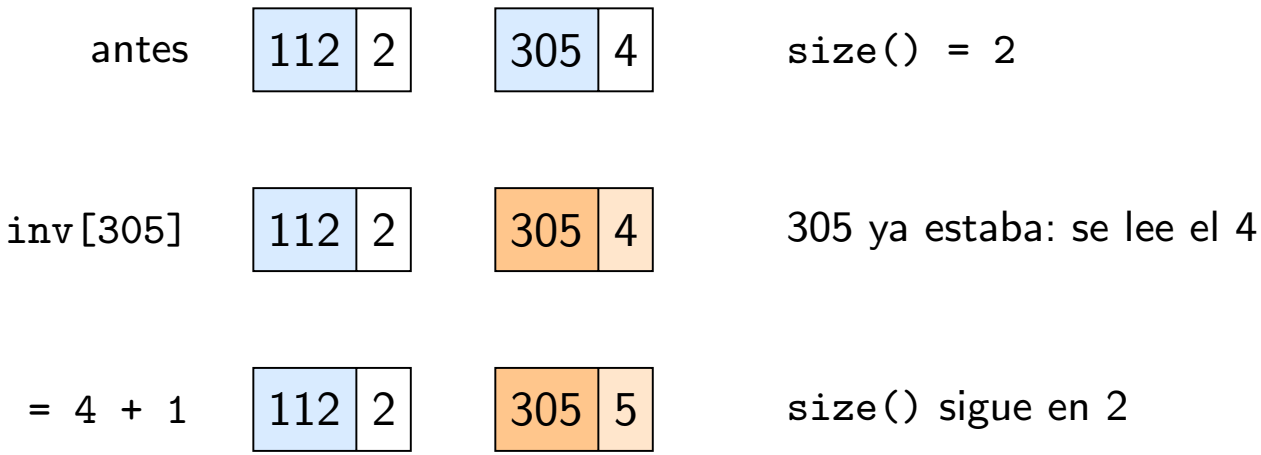
```
inv[112] = 2;
```




Antes, porque $112 < 305$. El `map` pone cada clave en su lugar al insertarla, sin correr a nadie como haría el vector.

# Insertar con []: una clave que ya está I

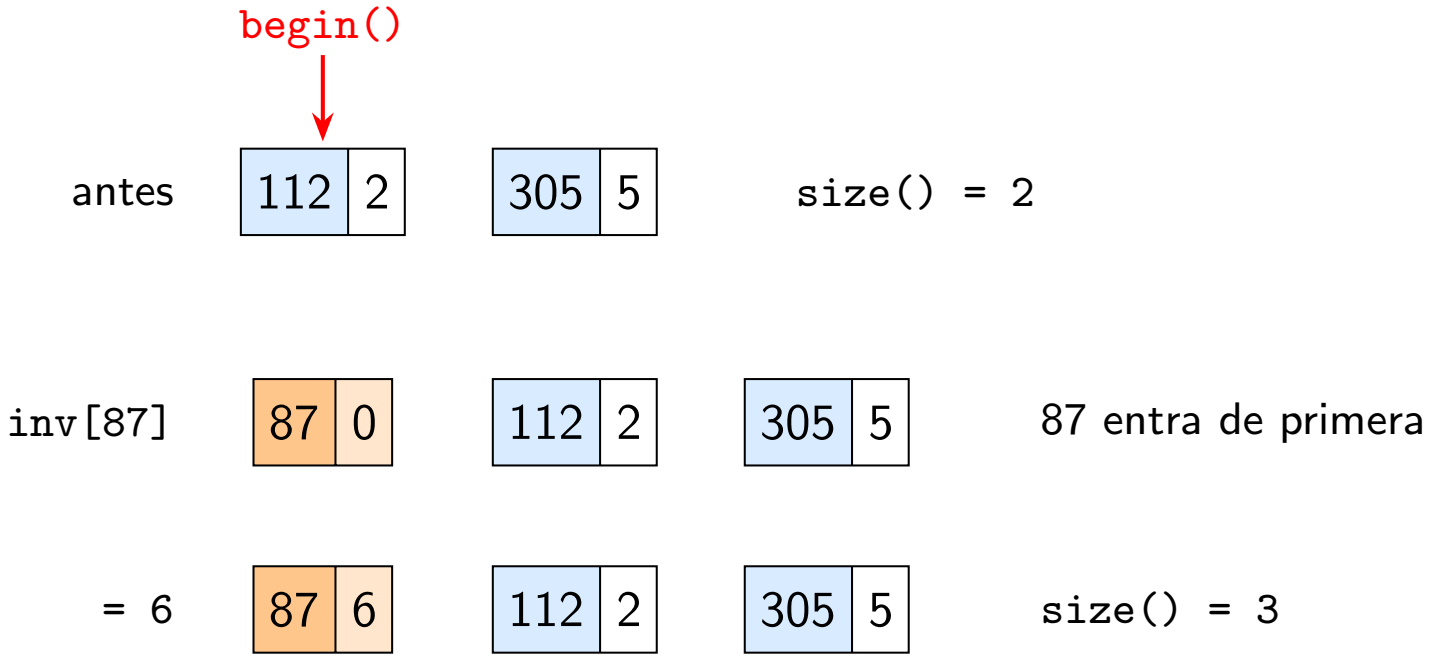
```
inv[305] = inv[305] + 1;
```




Con la clave presente, `inv[305]` no inserta: devuelve el valor que hay y la asignación lo cambia.

# Insertar con []: la nueva menor clave I

```
inv[87] = 6;
```




Antes de insertar, `begin()` apuntaba a 112. Ahora la menor clave es 87 y `begin()` apunta a ella.

# Eliminar: erase I

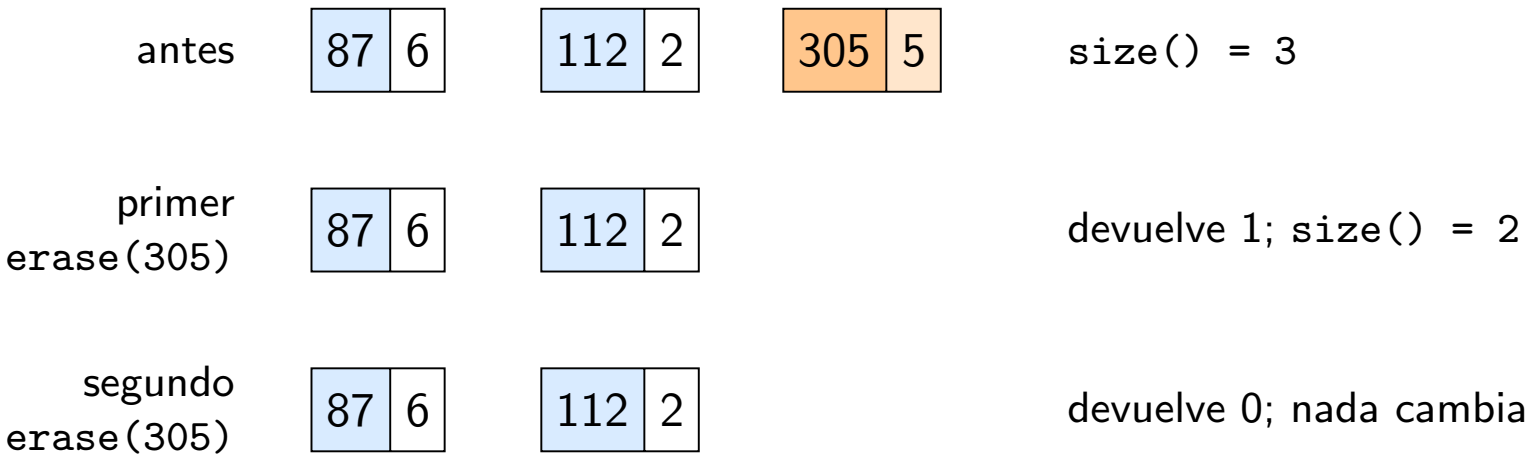
```
printf("%d\n", (int) inv.erase(305));
printf("%d\n", (int) inv.erase(305));
```




**Pregunta**

¿Qué imprime cada llamada, y cómo queda el `map`?

# Eliminar: `erase` II




La pareja de 305 sale entera, clave y valor. Sacar una clave que no está es válido y devuelve 0, así que `erase(k)` no necesita un `count` antes.

# La traza completa I

| Línea | Contenido, en orden | `size()` |
|---|---|---|
| `map<int, int> inv;` | (vacío) | 0 |
| `inv[305] = 4;` | (305, 4) | 1 |
| `inv[112] = 2;` | (112, 2) (305, 4) | 2 |
| `inv[305] = inv[305] + 1;` | (112, 2) (305, 5) | 2 |
| `inv[87] = 6;` | (87, 6) (112, 2) (305, 5) | 3 |
| `inv.erase(305);` | (87, 6) (112, 2) | 2 |

`size()` sube solo cuando entra una clave que no estaba, y baja solo con `erase`. Cambiar un valor no la mueve.

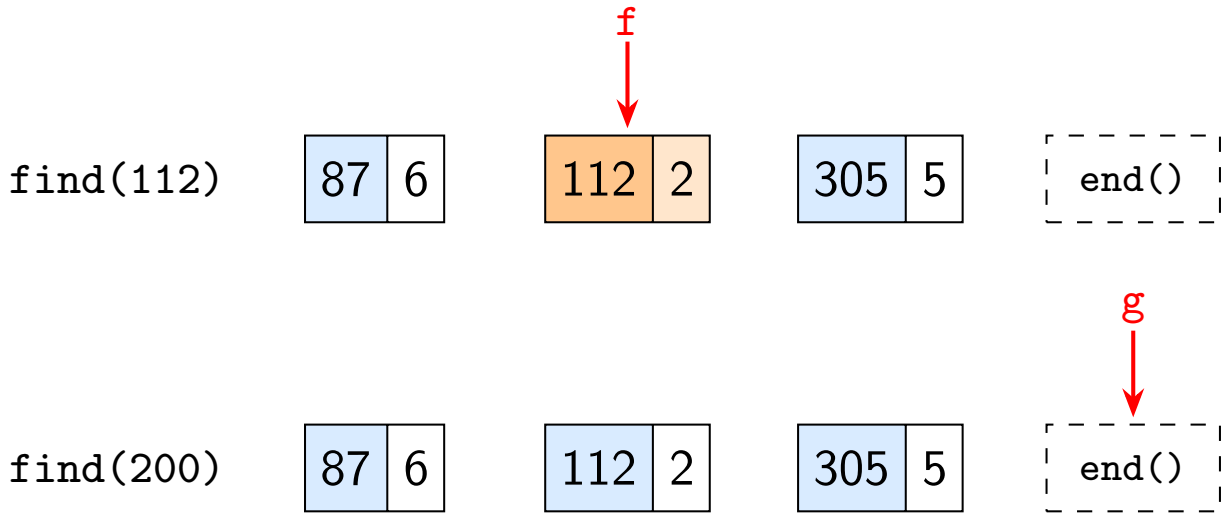
# Buscar: `find` I

El inventario tiene (87, 6) (112, 2) (305, 5).

**Pregunta**

1. Si `f = inv.find(112)`, ¿a qué apunta `f`, y cuánto valen `f->first` y `f->second`?
2. ¿Qué devuelve `inv.find(200)`?

# Buscar: `find` II




- `f` apunta a la pareja de 112: `f->first` es 112 y `f->second` es 2.
- 200 no está: `find` devuelve `inv.end()`, la posición después de la mayor clave, que no guarda ninguna pareja. Se compara con `end()` antes de leer `->second`.
- En los dos casos el `map` queda igual: `find` no inserta.

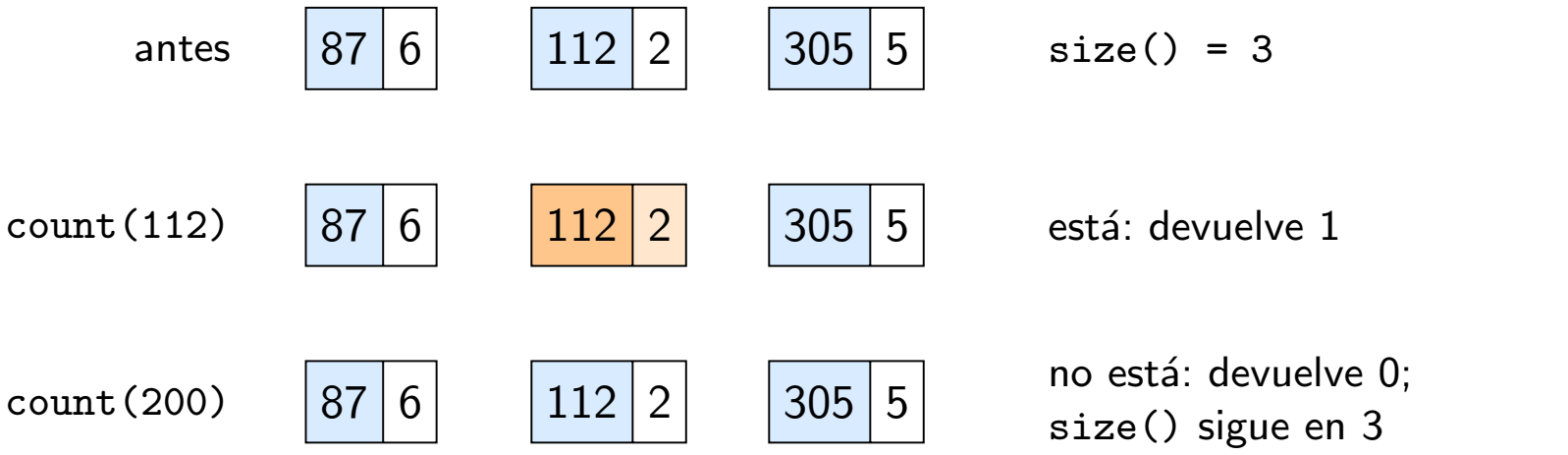
# Buscar: `find` III

```
map<int, int>::iterator f = inv.find(112);
if (f != inv.end()) {
  printf("find(112): clave %d, valor %d\n", f->first, f->second);
}
printf("find(200) == end(): %d\n", (int) (inv.find(200) ==
↪  inv.end()));
```

**Salida**

```
find(112): clave 112, valor 2 y find(200) == end(): 1.
```

# Contar: `count` I




Una clave aparece una vez o ninguna, así que `count` solo devuelve 1 o 0. Responde si está; `find` además deja en la mano la pareja, sin buscarla dos veces.

```
printf("count(112) = %d, count(200) = %d\n", (int) inv.count(112),
↪  (int) inv.count(200));
```

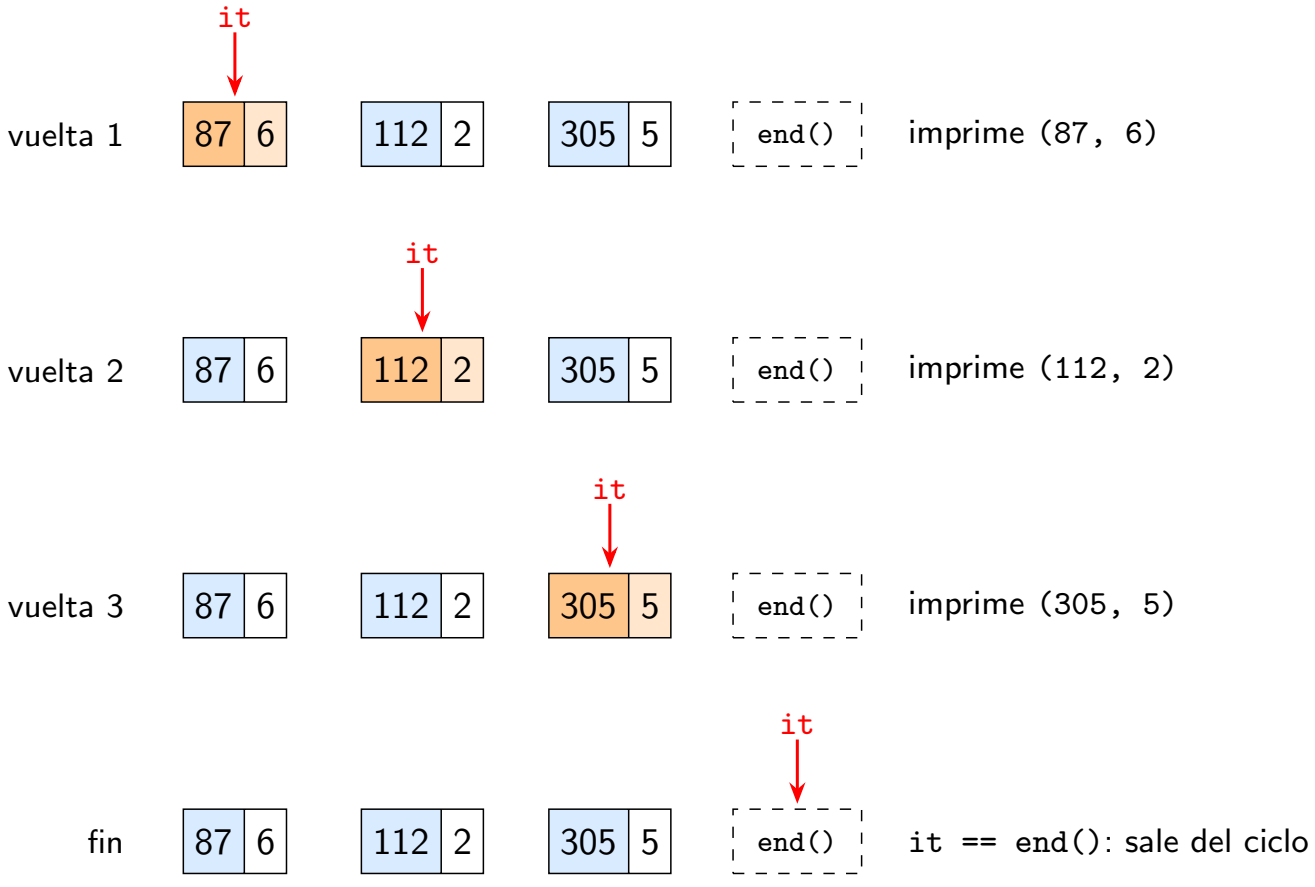
# Recorrer en orden I

El recorrido es el mismo de `vector` y `list`: un iterador desde `begin()` hasta `end()`. Lo nuevo es que cada posición guarda una pareja.

```
// recorre el map con su iterador: las claves salen de menor a mayor
void imprimir(map<int, int> &m) {
  map<int, int>::iterator it = m.begin();
  while (it != m.end()) {
    printf(" (%d, %d)", it->first, it->second);
    ++it;
  }
  printf("   size = %d\n", (int) m.size());
}
```

- `it->first` es la clave y `it->second` el valor.
- `++it` pasa a la siguiente clave mayor.

# Recorrer en orden II




Las claves salen de menor a mayor sin ordenar nada: el orden ya está en el `map`.

# Recorrer en orden III

Un programa hace las líneas de la traza, llama a `imprimir` después de cada una y termina con las consultas de `count`, `find` y `erase`:

**Salida**

```
 (305, 4)   size = 1
 (112, 2) (305, 4)   size = 2
 (112, 2) (305, 5)   size = 2
 (87, 6) (112, 2) (305, 5)   size = 3
count(112) = 1, count(200) = 0
find(112): clave 112, valor 2
find(200) == end(): 1
erase(305) = 1
erase(305) = 0
 (87, 6) (112, 2)   size = 2
```

Las cuatro primeras líneas y la última son filas de la traza.

# Preguntar con corchetes I

El inventario tiene (87, 6) (112, 5) (305, 5). Se consultan 200, 112 y 999 así:

```
while (i < 3) {
  int k = consultas[i];
  if (inv[k] != 0) {              // si k no estaba, esta linea lo
  ↪  inserta con 0
    printf("%d: hay %d\n", k, inv[k]);
  } else {
    printf("%d: no hay\n", k);
  }
  printf("   size = %d\n", (int) inv.size());
  i = i + 1;
}
```

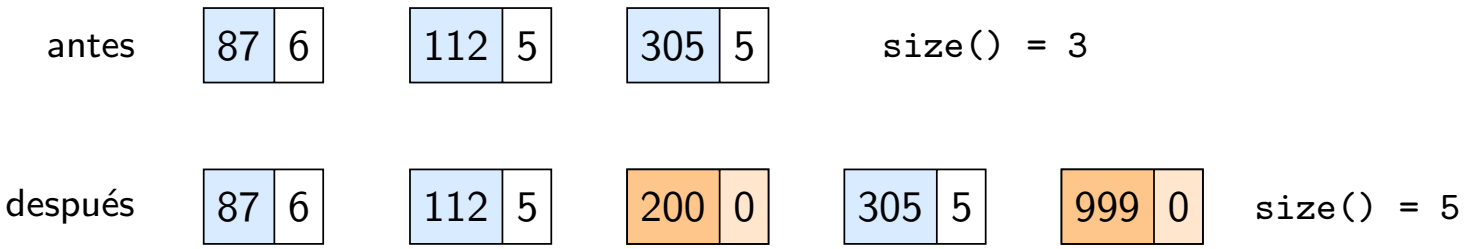
**Pregunta**

¿Qué imprime para cada consulta, y cuánto vale `size()` al final?

# Preguntar con corchetes II

## Salida

```
200: no hay, size = 4; 112: hay 5, size = 4; 999: no hay, size = 5.
```




Las respuestas son correctas, pero entraron (200, 0) y (999, 0), dos productos que nunca llegaron. Un recorrido los imprimiría con total 0.

## `m[k]` inserta

Si `k` no está, `m[k]` inserta $(k, 0)$ antes de devolver el valor. Para preguntar si una clave está se usa `count` o `find`, que no tocan el `map`.

# Preguntar con corchetes III

```
while (i < 3) {
  int k = consultas[i];
  if (otro.count(k) == 1) {
    printf("%d: hay %d\n", k, otro[k]);
  } else {
    printf("%d: no hay\n", k);
  }
  printf("   size = %d\n", (int) otro.size());
  i = i + 1;
}
```

# Preguntar con corchetes IV

Con `count`, las tres consultas imprimen lo mismo y `size` se queda en 3. Dentro del `if`, `otro[k]` ya no inserta nada: la clave está.

**La misma inserción, a favor**

En el inventario, `total[codigo] = total[codigo] + cantidad` funciona porque un código nuevo entra con 0. La inserción que estorba al consultar es la que hace funcionar el conteo.

# Claves que son palabras I

**Problema**

Leer palabras hasta el fin de la entrada e imprimir cada una con las veces que apareció, en orden alfabético.

**Pregunta**

Las palabras se leen con `scanf("%s", pal)` en un `char pal[101]`. ¿Sirve `map<char *, int>`, con `pal` como clave?

# Claves que son palabras II

## No sirve: compara direcciones

Con `map<char *, int>` la clave es un puntero, y dos punteros se comparan por la dirección, no por las letras. Todas las palabras se leen en el mismo arreglo `pal`, así que todas son la misma clave. Con la entrada `sol mar sol luna mar sol`, el `map` termina con una sola pareja, `sol 6`: la última palabra leída con la cuenta de las seis.

## La clave es un `string`

`string clave = pal;` copia las letras en un `string` propio, y dos `string` se comparan con `<` en orden alfabético, como `string("cola") < string("pila")`. Para imprimirlo con `printf` se usa `c_str()`.

# Claves que son palabras III

```cpp
int main() {
  map<string, int> veces;
  char pal[101];
  while (scanf("%s", pal) == 1) {
    string clave = pal;     // copia las letras: dos string se
    ↪  comparan por su texto
    veces[clave] = veces[clave] + 1;
  }
  map<string, int>::iterator it = veces.begin();
  while (it != veces.end()) {
    printf("%s %d\n", it->first.c_str(), it->second);
    ++it;
  }
  return 0;
}
```

# Claves que son palabras IV

**Con `sol mar sol luna mar sol`**

`luna` 1, `mar` 2, `sol` 3.

Comparar dos palabras de largo $L$ cuesta hasta $L$ pasos, así que cada operación con claves de texto cuesta $O(L \log n)$.

# El inventario, resuelto I

```cpp
int main() {
  int n;
  scanf("%d", &n);
  map<int, int> total;
  int i = 0;
  while (i < n) {
    int codigo, cantidad;
    scanf("%d %d", &codigo, &cantidad);
    total[codigo] = total[codigo] + cantidad;    // si no estaba,
    ↪  entra con 0
    i = i + 1;
  }
  map<int, int>::iterator it = total.begin();
  while (it != total.end()) {
    printf("%d %d\n", it->first, it->second);
    ++it;
  }
  return 0;
}
```

# El inventario, resuelto II

**Con los cinco registros del problema**

`87 6`, `112 5` y `305 5`.

- Cada registro hace una operación `[]` sobre un `map` de a lo sumo $n$ claves: $O(\log n)$. Con $n$ registros, $O(n \log n)$.
- El recorrido final visita las $d$ parejas una vez: $\Theta(d)$, con $d \leq n$ códigos distintos.
- Espacio: $\Theta(d)$, una pareja por código distinto.

# Contenido I


1 Repaso: la tabla hash con encadenamiento

2 Un problema donde importa el orden

3 El contenedor `map`

4 Lo que cuesta

5 Para el juez

6 Ejercicios

# ¿De dónde sale $\log n$? I

**Pregunta**

Un `map` tiene un millón de claves. ¿Cuántas comparaciones hace, más o menos, `find`?

# ¿De dónde sale $\log n$? II

Como la búsqueda binaria sobre un arreglo ordenado: cada comparación descarta cerca de la mitad de las claves que quedan.

| Comparaciones | 0 | 1 | 2 | 3 | $\cdots$ | 20 |
|---|---|---|---|---|---|---|
| Claves que quedan | $10^6$ | $5 \cdot 10^5$ | $2{,}5 \cdot 10^5$ | $1{,}25 \cdot 10^5$ | $\cdots$ | 1 |

$\log_2 10^6 \approx 19{,}9$: unas veinte comparaciones, frente al millón que haría una lista enlazada.

- El vector ordenado también busca así, pero insertar le cuesta $\Theta(n)$ por correr los elementos.
- El árbol balanceado del `map` mantiene buscar, insertar y eliminar en $O(\log n)$, también en el peor caso [1].

# Lo que cobra el `map` I

| Operación | Tiempo | Por qué |
|---|---|---|
| `m[k]`, `m[k] = v` | $O(\log n)$ | se busca `k`; si no está, entra en su lugar sin correr a nadie |
| `count(k)`, `find(k)` | $O(\log n)$ | la misma búsqueda, sin insertar |
| `erase(k)` | $O(\log n)$ | se busca `k` y se saca su pareja |
| `size()`, `empty()` | $\Theta(1)$ | el `map` lleva la cuenta |
| `begin()` | $\Theta(1)$ | la biblioteca tiene a mano la menor clave |
| recorrido completo | $\Theta(n)$ | cada pareja se visita una vez |
| destructor | $\Theta(n)$ | se libera un nodo por pareja |

## Espacio

$\Theta(n)$: un nodo por pareja, con la clave, el valor y los punteros de la estructura. Ninguna operación pide memoria aparte que crezca con $n$. Con claves de texto de largo $L$, cada comparación cuesta hasta $L$ y las operaciones pasan a $O(L \log n)$.

# Tabla hash frente a `map` I

| | Tabla hash, $m$ proporcional a $n$ | `map` |
|---|---|---|
| Buscar, insertar, eliminar | $\Theta(1+\alpha)$ promedio; $\Theta(n)$ peor | $O(\log n)$, también en el peor caso |
| Recorrer en orden de clave | $\Theta(m+n)$ y ordenar, $\Theta(n \log n)$ | $\Theta(n)$ |
| La menor clave | $\Theta(m+n)$, revisando todo | $\Theta(1)$ con `begin()` |
| Espacio | $\Theta(m+n)$ | $\Theta(n)$ |
| Lo que pide de la clave | una función hash $h$ | comparar con `<` |

## Pregunta

En promedio, $\Theta(1)$ le gana a $O(\log n)$. ¿Por cuánto, con $n = 10^5$ y $n = 10^6$?

# Tabla hash frente a map II

```
void medirMap(int n) {
  clock_t ini = clock();
  map<int, int> mp;
  int i = 0;
  while (i < n) {
    mp[(int) ((long long) i * 7919 % P)] = i;
    i = i + 1;
  }
  int hallados = 0;
  i = 0;
  while (i < n) {
    if (mp.count((int) ((long long) i * 7919 % P)) == 1) {
      hallados = hallados + 1;
    }
    i = i + 1;
  }
  double s = (double) (clock() - ini) / CLOCKS_PER_SEC;
  printf("map,  n = %d: %.3f s (%d hallados)\n", n, s, hallados);
}
```

# Tabla hash frente a `map` III

Un programa inserta $n$ claves distintas y después las busca todas, primero en la tabla hash, con $m$ primo cercano a $n$, y después en un `map`. Esta es la parte del `map`; `medirHash` es igual, con `TablaHash t(m)`, `insertar` y `buscar`.

# Tabla hash frente a `map` IV

Tiempo de procesador de las $n$ inserciones y las $n$ búsquedas, compilado con `g++ -Wall -Wextra`, sin optimizaciones, en el equipo donde se preparó la sesión:

| $n$ | $m$ | Tabla hash | `map` |
|---|---|---|---|
| $10^5$ | 99 991 | 0,005 s | 0,068 s |
| $10^6$ | 999 983 | 0,076 s | 0,780 s |

- En los dos tamaños la tabla hash es entre diez y catorce veces más rápida para buscar e insertar.
- El `map` entrega a cambio el orden: el recorrido sale de menor a mayor sin pagar un ordenamiento.

# ¿Cuál escoger? I

## Tabla hash

Cuando solo se pregunta si una clave está y qué valor tiene, con muchas consultas, y el orden de las claves no aparece en la salida.

## `map`

Cuando la salida va en orden de clave, cuando se pide la menor o la mayor clave en cualquier momento, o cuando no hay a mano una buena función hash para la clave y sí una comparación $<$.

## El peor caso

La tabla hash depende de que $h$ reparta bien: con una mala $m$ o con claves hechas a su medida, buscar llega a $\Theta(n)$. El `map` se queda en $O(\log n)$ con cualquier orden de llegada de las claves.

# Contenido I


1 Repaso: la tabla hash con encadenamiento

2 Un problema donde importa el orden

3 El contenedor `map`

4 Lo que cuesta

5 Para el juez

6 Ejercicios

# Para el juez: UVa 11286: Conformity I

**UVa 11286: Conformity**,
`https://onlinejudge.org/external/112/11286.pdf`
Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=2261`

## Qué pide

Varios casos, terminados por una línea con 0. Cada caso trae $n$ estudiantes, $1 \leq n \leq 10\,000$, y cada estudiante escoge cinco cursos distintos, numerados entre 100 y 499. La popularidad de una combinación es cuántos estudiantes escogieron exactamente esos cinco cursos. Se imprime cuántos estudiantes tienen una combinación de popularidad máxima.

# Para el juez: UVa 11286: Conformity II

## ¿Entendimos el problema?

En el primer caso del enunciado, el primer y el tercer estudiante escogen 100, 101, 102, 103 y 488 en distinto orden: esa combinación tiene popularidad 2 y la respuesta es 2. En el segundo caso las tres combinaciones son distintas y cada una tiene popularidad 1. ¿Por qué la respuesta es 3 y no 1?

Las tres empatan en el máximo, y cuentan los estudiantes de todas.

## Cómo atacarlo

Se ordenan los cinco cursos con `sort` y se pegan en un número de 15 cifras, tres por curso: esa es la clave de un `map<long long, int>` que cuenta. Al final, un recorrido halla la popularidad mayor y suma los estudiantes de las combinaciones que la alcanzan. Cuesta $O(n \log n)$ por caso y $\Theta(n)$ de espacio. El orden de las claves no se usa: la tabla hash también sirve, con una $h$ para claves de 15 cifras.

# Para el juez: UVa 11286: Conformity III

## Las trampas

- El orden de los cursos en la línea no importa. Sin ordenar, `100 101 102 103 488` y `103 102 101 488 100` serían dos claves distintas.
- Se imprime el total de estudiantes en las combinaciones empatadas en el máximo, no cuántas combinaciones son.
- La clave no cabe en `int`: 499 499 499 499 499 pide `long long`, que se lee e imprime con `%lld`.

# Para el juez: UVa 10295: Hay Points I

**UVa 10295: Hay Points**,
`https://onlinejudge.org/external/102/10295.pdf`
Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=1236`

## Qué pide

Un diccionario de $m \leq 1000$ palabras, de hasta 16 letras minúsculas, cada una con un valor en dólares, un número real entre 0 y 1 000 000. Después vienen $n \leq 100$ descripciones de cargo, de una o más líneas cada una, terminadas por una línea con un punto. Por cada descripción se imprime la suma de los valores de todas las palabras que aparecen en ella; las que no están en el diccionario valen 0.

# Para el juez: UVa 10295: Hay Points II

## ¿Entendimos el problema?

En la primera descripción del enunciado aparecen `administer` (100 000), `spending` dos veces (400 000), `money` dos veces (150 000), `responsibility` dos veces (50 000), `expertise` (100) y `skill` (50): 700 150. ¿Cuánto vale `managing`?

Cero: el diccionario tiene `manage`, y la palabra tiene que coincidir completa.

## Cómo atacarlo

El diccionario va en un `map<string, double>`. Cada descripción se lee palabra por palabra con `scanf` hasta la palabra `.`, que se detecta con `strcmp`, y cada palabra se consulta con `find`. Con $w$ palabras en una descripción, cuesta $O(w \log m)$ comparaciones de palabras.

# Para el juez: UVa 10295: Hay Points III

## Las trampas

- Consultar con `valor[pal]` inserta con 0 cada palabra que no está. La suma no cambia, pero el diccionario crece con cada palabra de las descripciones y las búsquedas se alargan. `find` no inserta.
- El valor es un número real: se lee como `double` con `%lf`. El ejemplo imprime enteros, y la suma se imprime con `%.0f`.
- Una descripción ocupa varias líneas: se lee por palabras, no por líneas, y termina en la palabra que es solo un punto.

# Contenido I


1 Repaso: la tabla hash con encadenamiento

2 Un problema donde importa el orden

3 El contenedor `map`

4 Lo que cuesta

5 Para el juez

6 Ejercicios

# Ejercicio en parejas: a mano I

```
map<int, int> m;
m[40] = 3;
m[15] = 8;
m[72] = 1;
m[15] = m[15] - 8;
m.erase(72);
m.erase(99);
if (m[60] == 0) {
  m[33] = 2;
}
```

## Las preguntas

1. ¿Qué parejas quedan, en el orden en que las recorre un iterador?
2. ¿Cuánto vale `m.size()`?
3. ¿Qué línea inserta una clave a la que nadie le asigna valor?

# Ejercicio en parejas: a mano II

**Lo que debe salir**

1. (15, 0) (33, 2) (40, 3) (60, 0).
2. 4.
3. La condición del `if`: `m[60]` inserta (60, 0) para poder compararlo con 0. Como da 0, entra también (33, 2).

- Dejar el valor de 15 en 0 no lo saca: sigue en el `map` hasta un `erase(15)`.
- `m.erase(99)` devuelve 0 y no cambia nada.

# Ejercicio en parejas: quiénes faltaron I

Se tienen los códigos de los $n_a$ inscritos de un curso y los de los $n_b$ que asistieron. Hay que imprimir los que faltaron.

## Las preguntas

1. Con inscritos $\langle 4021, 3307, 5590, 1288, 4410, 2765 \rangle$ y asistentes $\langle 5590, 4021, 4410 \rangle$, ¿quiénes faltaron?
2. Escriban `faltaronHash` con la tabla hash de la sesión de tablas hash, en el orden de la lista de inscritos. ¿Cuánto cuesta?
3. Ahora la lista sale de menor a mayor código. Escriban `faltaronMap`. ¿Cuánto cuesta?

# Ejercicio en parejas: quiénes faltaron II

```
// en el orden de la lista de inscritos: n_b inserciones y n_a
↪  busquedas, O(n_a + n_b) en promedio
void faltaronHash(int a[], int na, int b[], int nb) {
  TablaHash vinieron(11);
  int i = 0;
  while (i < nb) {
    vinieron.insertar(b[i]);
    i = i + 1;
  }
  i = 0;
  while (i < na) {
    if (!vinieron.buscar(a[i])) {
      printf(" %d", a[i]);
    }
    i = i + 1;
  }
  printf("\n");
}
```

# Ejercicio en parejas: quiénes faltaron III

```
// de menor a mayor codigo: n_a + n_b operaciones de O(log n_a)
void faltaronMap(int a[], int na, int b[], int nb) {
  map<int, int> falta;
  int i = 0;
  while (i < na) {
    falta[a[i]] = 1;
    i = i + 1;
  }
  i = 0;
  while (i < nb) {
    falta.erase(b[i]);
    i = i + 1;
  }
  map<int, int>::iterator it = falta.begin();
  while (it != falta.end()) {
    printf(" %d", it->first);
    ++it;
  }
  printf("\n");
}
```

# Ejercicio en parejas: quiénes faltaron IV

## Lo que debe salir

1. 3307, 1288 y 2765. La tabla hash imprime `3307 1288 2765`, en el orden de los inscritos.
2. $n_b$ inserciones y $n_a$ búsquedas de $\Theta(1+\alpha)$: $\Theta(n_a + n_b)$ en promedio, con $m$ proporcional a $n_b$.
3. El `map` imprime `1288 2765 3307`. Son $n_a$ inserciones y $n_b$ llamadas a `erase`, cada una $O(\log n_a)$: $O((n_a + n_b)\log n_a)$.

`erase` de un asistente que no estaba inscrito devuelve 0 y no hace nada: no hace falta preguntar antes.

# Ejercicio en parejas: la moda I

La moda de un arreglo es el valor que más se repite. Si hay empate, se responde el menor.

## Las preguntas

1. Con $a = \langle 5, 2, 8, 2, 5, 9, 5, 2 \rangle$, ¿qué responde?
2. Escriban `moda` con un `map<int, int>`. ¿Por qué basta comparar con > estricto para resolver el empate?
3. ¿Qué cambia si se cuenta con la tabla hash?

# Ejercicio en parejas: la moda II

```
int moda(int a[], int n) {
  map<int, int> veces;
  int i = 0;
  while (i < n) {
    veces[a[i]] = veces[a[i]] + 1;
    i = i + 1;
  }
  int mejor = 0;
  int cuantas = 0;
  map<int, int>::iterator it = veces.begin();
  while (it != veces.end()) {
    if (it->second > cuantas) {     // con > estricto, en un empate
    ↪  se queda la menor
      mejor = it->first;
      cuantas = it->second;
    }
    ++it;
  }
  return mejor;
}
```

# Ejercicio en parejas: la moda III

## Lo que debe salir

1. 2. El 2 y el 5 aparecen tres veces cada uno, y 2 es el menor.
2. El recorrido va de menor a mayor clave: la primera clave que alcanza la cuenta mayor es la menor de las empatadas, y con > estricto las siguientes no la reemplazan.
3. La tabla recorre por casillas, no por clave. En el empate hay que comparar también las claves, y cada nodo tiene que guardar su cuenta. Cuesta $\Theta(n)$ en promedio, frente a $O(n \log n)$ del `map`.

# ¿Tabla hash o `map`? I

Para cada situación, ¿qué estructura escogen, y por qué?

1. Un corrector revisa un millón de palabras contra un diccionario de cien mil y solo dice si cada una está.
2. Una agenda imprime los contactos en orden alfabético después de cada cambio.
3. Un sistema de turnos pide, en cualquier momento, el código más pequeño registrado.
4. Se cuenta cuántas veces aparece cada número de una lista de un millón para imprimir los cinco más frecuentes.

# ¿Tabla hash o `map`? II

**Respuestas**

1. Tabla hash: un millón de búsquedas y ningún orden en la salida.
2. `map`: el recorrido ya sale en orden, $\Theta(n)$ por impresión, sin ordenar cada vez.
3. `map`: `begin()` da la menor clave en $\Theta(1)$.
4. Tabla hash. La salida se ordena por frecuencia, no por clave, así que el orden del `map` no ahorra nada.

# Propuestos para la casa I

1. **Entradas y salidas.** Cada registro del inventario trae un tipo, 1 para entrada y 2 para salida, un código y una cantidad. Un código cuyo total llega a 0 desaparece. Al final se imprimen los totales en orden de código.
2. **Distintos en orden.** Imprimir los valores distintos de un arreglo de menor a mayor, una vez con un `map` y otra con la tabla hash y `sort`. ¿Cuánto cuesta cada versión?
3. **Anagramas.** Decidir si dos palabras usan las mismas letras el mismo número de veces, con un `map<char, int>`. Con solo letras minúsculas, ¿qué estructura de la sesión de tablas directas sirve mejor?
4. **Hay Points con la tabla hash.** Resolver UVa 10295 con una tabla hash de palabras cuyo nodo guarda la palabra y su valor.
5. **Promedio por estudiante.** Cada registro trae un código y una nota en décimas. Imprimir, en orden de código, el promedio de cada estudiante.

# Pistas para los propuestos I

1. Se suma o se resta con `[]`; si el total queda en 0, se llama a `erase`. $O(\log n)$ por registro.
2. Con el `map`, se insertan los valores y se recorre: $O(n \log n)$. Con la tabla, se inserta cada valor que no estaba, se copian los distintos a un arreglo y se ordena: $\Theta(n)$ en promedio más $\Theta(d \log d)$ para los $d$ distintos.
3. Se suma uno por cada letra de la primera y se resta uno por cada letra de la segunda; son anagramas si todos los valores terminan en 0. Con 26 letras, una tabla directa de 26 casillas hace cada paso en $\Theta(1)$.
4. La casilla sale del polinomio en base 31 y la comparación de la lista pasa a `strcmp`. Las palabras que no están no se insertan.
5. Dos `map<int, int>`, uno con la suma y otro con la cantidad de notas, o uno solo cuyo valor sea un `struct` con las dos.

# Reto de cierre I

## Claves en un rango

Un `map<int, int>` tiene $n$ claves. Dados $a \leq b$, hay que imprimir las claves que están entre $a$ y $b$. Recorrer todo el `map` cuesta $\Theta(n)$ aunque en el rango haya dos claves. ¿Se puede pagar algo cercano a $\log n$ más lo que se imprime? ¿Qué operación le faltaría al contrato para lograrlo?

# Resumen I

- La tabla hash recorre por casillas: imprimir en orden de clave le exige ordenar.
- Un `map<K, V>` guarda parejas con claves distintas en orden creciente: `m[k]` lee o asigna, `count` y `find` consultan, `erase` saca, y un iterador recorre de la menor clave a la mayor.
- `m[k]` inserta $(k, 0)$ si `k` no estaba: sirve para contar y estorba para preguntar.
- Una clave de texto va como `string`; un `char *` se compara por dirección.
- Buscar, insertar y eliminar cuestan $O(\log n)$ también en el peor caso; recorrer, $\Theta(n)$. Con un millón de claves la tabla hash fue unas diez veces más rápida, y el `map` se escoge cuando el orden de las claves está en la salida.

**Lo que sigue**

Los árboles: la estructura con la que la biblioteca guarda el `map`, y de donde sale su $O(\log n)$.

# Referencias I




T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022. Capítulo 11, tablas hash (§11.2, encadenamiento y su análisis); capítulo 12, árboles binarios de búsqueda; capítulo 13, árboles rojinegros, donde buscar, insertar y eliminar cuestan $O(\log n)$.



cppreference.com. *std::map*: `https://en.cppreference.com/w/cpp/container/map`. Contrato y complejidad de cada operación.



UVa Online Judge. Problema 11286, *Conformity*: `https://onlinejudge.org/external/112/11286.pdf`.



UVa Online Judge. Problema 10295, *Hay Points*: `https://onlinejudge.org/external/102/10295.pdf`.